[Normal] Delete runtime Metadata CRs without an AppDeployment (Negative Rings)

By José Vieira In progress 3 min 10 Add a reaction

To request Emergency/Normal Product Infrastructure Changes [ODC + other services] please follow the work instructions.

https://outsystemsrd.atlassian.net/wiki/spaces/RKB/pages/3862331705/Change+Management+Process#Record-Request-for-Changes

Please, find below the **Template to Document a New Product Infrastructure Change [ODC + other services].**

Feel free to copy this page to your workspace and update the title not forgetting to:

- eliminate the [TEMPLATE] brackets,
- **Always** indicate the type of change [Standard/Normal/Emergency] in the title
- replace the * Document a New Change * with the title that better reflects what is the change you are documenting.

This Change is part of the https://outsystemsrd.atlassian.net/wiki/spaces/RKB/pages/3798631729.

1. Summary

As part of the initiative of the AI Agent Builder, the AIAB Platform Service is generating for each application that is published the application metadata and sending it to runtime via NATS2CRD to create the metadata CR

ma%20Metadata

Due to a flaw in the design when an application was deleted the corresponding metadata CR for that application was not being deleted, this lead to the situation of taking space and raising alerts in the backup, as described in the following slack thread and in this incident:

- https://outsystemsrd.atlassian.net/browse/RDINC-44164

2. General Information

Change Type	NORMAL
Approved Normal RFC *Applicable only for Standard Changes*	*<Jira link of the Normal Request for Change submitted to approve the promotion of this change from Normal to Standard Change>*
Change Risk	LOW https://outsystemsrd.atlassian.net/wiki/spaces/RKB/pages/3805708366
Downtime Involved	NO
Rollback Available	NO
Implementation Team	SRE
Component(s)/Asset(s)	N/A
Impact Description	No impact expected

Inputs for execution

- Power user access to each negative ring to AWS runtime stamp

Technical requirements

- Command line `kubectl` installed
- `bash` or compatible shell

Communication

- No communication needed.

Expected duration

- ~~30 minutes~~
- Depends on the total number of CR inside the cluster

4. Implementation Plan

Implementation actions

Repeat the following operations for all the AWS Runtime accounts clusters:

- Ring -3
  - `arn:aws:eks:us-east-1:457833322001:cluster/rundev-dev-us-east-1-01`
  - `arn:aws:eks:us-east-1:457833322001:cluster/runnp-dev-us-east-1-01`
  - `arn:aws:eks:us-east-1:457833322001:cluster/runp-dev-us-east-1-01`
- Ring -2
  - `arn:aws:eks:us-east-1:974136907260:cluster/rundev-test-us-east-1-01`
  - `arn:aws:eks:us-east-1:974136907260:cluster/runnp-test-us-east-1-01`
  - `arn:aws:eks:us-east-1:974136907260:cluster/runp-test-us-east-1-01`

east-1-01

- arn:aws:eks:us-east-1:768592925077:cluster/runnp-stage-us-east-1-01
- arn:aws:eks:us-east-1:768592925077:cluster/runp-stage-us-east-1-01
- arn:aws:eks:ap-southeast-1:861089571927:cluster/rundev-stage-ap-southeast-1-01
- arn:aws:eks:ap-southeast-1:861089571927:cluster/runnp-stage-ap-southeast-1-01
- arn:aws:eks:ap-southeast-1:861089571927:cluster/runp-stage-ap-southeast-1-01

1. First get access to the ring by following this guide → https://outsystemsrd.atlassian.net/wiki/spaces/RKB/pages/3239608332/How+To+Access+the+Rings+Infrastructure#Get-Elevated-Permission
2. Then, follow this section of the same guide so that we'll be able to have access to the runtime-stamp cluster through kubectl → https://outsystemsrd.atlassian.net/wiki/spaces/RKB/pages/3239608332/How+To+Access+the+Rings+Infrastructure#Using-kubectl
3. Now run the following command to start cleaning up the metadata CRs without an ApplicationDeployment CR:

```
#!/bin/bash

# --- Function to select Kubernetes Context ---
select_kube_context() {
    echo "Fetching available Kubernetes contexts..."
    local contexts
    contexts=$(kubectl --context="$SELECTED_KUBE_CONTEXT" con

    if [ -z "$contexts" ]; then
        echo "Error: No Kubernetes contexts found. Please che
        exit 1
    fi

    echo ""
    echo "Available Kubernetes Contexts:"
    select context_name in $contexts "Exit"; do
```

```
        elif [ -n "$context_name" ]; then
            echo "Selected context: '$context_name'"
            echo ""
            SELECTED_KUBE_CONTEXT="$context_name"
            break
        else
            echo "Invalid selection. Please try again."
        fi
    done
}

# Call the function to prompt for context selection
select_kube_context

# Ensure a context was selected before proceeding
if [ -z "$SELECTED_KUBE_CONTEXT" ]; then
    echo "No context selected. Exiting."
    exit 1
fi

echo "Operating on Kubernetes context: '$SELECTED_KUBE_CONTEX

echo "Fetching all AppMetaData..."
# Get all AppMetaData names and namespaces as JSON
ALL_APPMETADATA=$(kubectl --context="$SELECTED_KUBE_CONTEXT"

echo "Fetching all ApplicationDeployments..."
# Get all ApplicationDeployment names and namespaces as JSON
ALL_APPDEPLOYMENTS=$(kubectl --context="$SELECTED_KUBE_CONTEX

# Create a set of "ApplicationDeployment" names for quick loo
# Format: "namespace/name"
APPDEPLOYMENT_SET=$(echo "$ALL_APPDEPLOYMENTS" | \
  jq -r '.items[] | "\(.metadata.namespace)/\(.metadata.name)

# Initialize arrays to store AppMetaData to delete and their
declare -a APPMETADATA_TO_DELETE_IDENTIFIERS
declare -a APPMETADATA_TO_DELETE_NAMESPACES

echo "Identifying orphan AppMetaData..."
# Iterate over AppMetaData and check if a corresponding Appli
# using the in-memory set.
while IFS=$'\n' read -r app_metadata_item; do
  app_metadata_name=$(echo "$app_metadata_item" | jq -r '.met
  app_metadata_namespace=$(echo "$app_metadata_item" | jq -r

  target_key="${app_metadata_namespace}/${app_metadata_name}"
```

```
    echo "   >>> Using context: $SELECTED_KUBE_CONTEXT"
    echo "   >>> AppMetaData '$app_metadata_name' in namespac
    APPMETADATA_TO_DELETE_IDENTIFIERS+=("appmetadata.charts.o
    APPMETADATA_TO_DELETE_NAMESPACES+=("$app_metadata_namespa
  fi
done < <(echo "$ALL_APPMETADATA" | jq -c '.items[]') # -c for

if [ ${#APPMETADATA_TO_DELETE_IDENTIFIERS[@]} -eq 0 ]; then
  echo "No orphaned AppMetaData found."
else
  echo "Found ${#APPMETADATA_TO_DELETE_IDENTIFIERS[@]} orphan

  NUM_PARALLEL_JOBS=8

  # Combine identifiers and namespaces for xargs.
  for i in "${!APPMETADATA_TO_DELETE_IDENTIFIERS[@]}"; do
    printf "%s %s\n" "${APPMETADATA_TO_DELETE_IDENTIFIERS[$i]
  done | xargs -P "$NUM_PARALLEL_JOBS" -I {} bash -c 'IFS=" "

  echo "Deletion process completed."
fi
```

Testing and validation plan

Once the command has finished is expected to have some visualization of the deleted metadata CRs

```
   >>> AppMetaData 'dd803b7b-dc10-49dd-8e94-8cca5b321382' in name
```

On each cluster confirm at least one of the deleted metadata CR doesn't exist in fact, for example:

```
kubectl get appmetadata.charts.outsystems.com dd803b7b-dc10-49dd-
```

Is expected to receive:

```
Error from server (NotFound): appmetadata.charts.outsystems.com "
```

No rollback plan

5. Post Implementation

None

Related content Send feedback

RFC - Remove ai-dummy-app orphan resources in ring plus3
Luís Filipe Silva

Remove upbound kms and alias CRs in negative rings
António Salgado

ICE | Change - Delete orphan CR
Ricardo Gonçalves

ICE | Change - Delete environment CR in stage
Daniel Rodrigues

ICE | Change - Clean up tenant on negative rings
Renato Torres

[Normal] - Get PowerUser access to negative rings (-2 and -1) to clean up test IPFilter CRs
Miguel Piedade